

SESSION 2: FEDERATED LEARNING SYSTEMS

What is Federated Learning

- Decentralized training across many edge devices or clients
- Data stays on device; only model updates or gradients are shared

Why it matters

- Privacy preservation and regulatory compliance
- Lower central bandwidth use; clients compute locally

Core challenges

- Clients are intermittent and heterogeneous (CPU/GPU, battery, network)
- Non-IID data across clients; drift and imbalance
- Faults and stragglers; limited communication rounds

FedAvg (Federated Averaging) protocol

- Round t :
 - Server broadcasts current model to selected clients
 - Clients train locally for a few epochs on their data
 - Clients send model deltas or updated weights to server
 - Server aggregates (weighted average by client data size) $\rightarrow$ new global model

Practical tips

- Client sampling: pick a subset each round to scale and reduce stragglers
- Weighting: average by number of local examples to reduce bias
- Compression: quantize/sparsify updates to save bandwidth
- Security/privacy: secure aggregation, differential privacy, device attestation
- Evaluation: track both global and per-client metrics; handle skewed datasets

Communication- Efficient Protocols

Core idea: cut bytes per round without hurting accuracy

Techniques

- **Quantization:** send updates in 8/4/2 bits with per-tensor scales
- **Sparsification:** top-k or thresholded updates; error feedback to recover dropped mass
- **Structured updates:** layer-wise or last-layer only; low-rank adapters (LoRA/PEFT)
- **Delta/periodic sync:** send changes since last round; skip small updates

Practicalities

- Clip and add DP noise if required; secure aggregation friendly
- Calibrate k/bit-width; monitor loss drift and add error feedback
- Compress after aggregation when possible to reduce server load